CAD for VLSI Design

指導教授: 陳聿廣

PA3

Graph-based STA

(結報)

系級 電機四 B

學號 110501544

姓名 翁瑄妤

目錄

1. Compilation and Execution 3

2. Completion 3

2.1 The Result of Execution 3

2.2 The Runtimes for the Public Test Cases 4

3. Explanation 4

3.1 Overall Framework 4

3.2 Data Structure Design 5

4. Method of Your Design 7

4.1 Overall Flow Chart 7

4.2 Core Algorithms 7

4.3 Implementation Details 10

5. Hardness 17

6. Suggestions 17

1. Compilation and Execution

主要在工作站上執行程式，其系統為 Linux 3.10.0、Host 為 140.115.71.87。

執行指令分為以下步驟:

1. 打開所屬資料夾 (所需資料皆在 PA3 的資料夾裡)

   cd PA3/

2. 執行 Makfile 檔案(有另外加 run1、run2、check1、check2 做測試用)：

```makefile
# PA3 cpp compiler
.PHONY: all run main run1 run2 check1 check2 clean
all: main.cpp parser.cpp calculate.cpp
	g++ -std=c++11 main.cpp parser.cpp calculate.cpp -o exe
run:
	./exe $(input) test_lib.lib
run1:`
	./exe c17.v test_lib.lib
run2:
	./exe example.v test_lib.lib
check1:
	./PA3_checker c17.v test_lib.lib  Load_110501544_c17.txt Delay_110501544_c17.txt Path_110501544_c17.txt
check2:
	./PA3_checker example.v test_lib.lib  Load_110501544_example.txt Delay_110501544_example.txt Path_110501544_example.txt
clean:
	rm exe
```

   - 將作業 cpp 檔輸出成執行檔.o 檔

     make all

   - 執行輸出檔，生成輸出檔案(以 c17 為例)

     make run input=c17.v

   - 清除執行檔

     make clean

3. 執行 CHECKER 檔確認輸出檔案內容是否正確 (以 c17 為例)

   chmod +x PA3_checker

   ./ PA3_checker c17.v test_lib.lib Load_110501544_c17.txt Delay_110501544_c17.txt Path_110501544_c17.txt

2. Completion

2.1 The Result of Execution

```
root@DESKTOP-6QUNLQQ:/mnt/c/Users/User/Desktop/CAD/PA3# make check1
./PA3_checker Opencase/c17.v Opencase/test_lib.lib  Load_110501544_c17.txt Delay_110501544_c17.txt Path_110501544_c17.txt
Step 1 Correct!!!
Step 2 Correct!!!
Step 3-1 Correct!!!
Step 3-2 Correct!!!
Score: 8
```

```
root@DESKTOP-6QUNLQQ:/mnt/c/Users/User/Desktop/CAD/PA3# make check2
./PA3_checker Opencase/example.v Opencase/test_lib.lib  Load_110501544_example.txt Delay_110501544_example.txt Path_110501544_example.txt
Step 1 Correct!!!
Step 2 Correct!!!
Step 3-1 Correct!!!
Step 3-2 Correct!!!
Score: 8
```

由上面兩張照片可得知，執行出來的兩份輸出檔案皆為正確，且指令皆以比照投影片要求執行，因此這份檔案已完成作業中的所有要求。

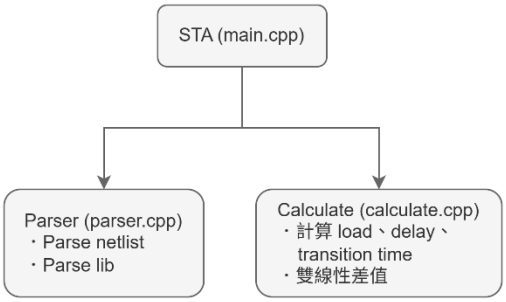
2.2 The Runtimes for the Public Test Cases

Testcase	Runtime (microeseconds)
c17.v	2881
example.v	2983

```
[s110501544@ncuee ~/PA3]$ make run input=c17.v
./exe c17.v test_lib.lib

Execution Time: 2881 microseconds
[s110501544@ncuee ~/PA3]$ make run input=example.v
./exe example.v test_lib.lib

Execution Time: 2983 microseconds
```

3. Explanation

3.1 Overall Framework



整體架構利用 class 模組化設計，主要包含以下幾個模組：

1. Parser 模組
   - 解析 netlist 檔案
   - 讀取 Liberty 格式的時序庫
   - 建立電路的內部資料結構

```
class Parser {
private:
    vector<double> Store_Value(string line);  // Extract values from a LUT line

// Public data for STA to access
public:
    // netlist information
    string module_name;
    unordered_map<string, Gate> gate_map;   // Gate name to Gate mapping
    unordered_map<string, Net> net_map;     // Net name to Net mapping
    vector<string> output_nets;             // Primary output list

    // library information
    vector<double> output_cap;              // Index_1 (capacitance values)
    vector<double> input_trans;             // Index_2 (input transition values)
    vector<CellLib> cell_lib;               // All parsed cell types

    // Parse the netlist and library files
    void Netlist_Parse(const string& filename);     // Parse netlist file
    void Lib_Parse(const string& filename);         // Parse library file
};
```

2. Calculate 模組
   - 計算 gate 間的輸出負載
   - 使用雙線性插值計算 delay 和 transition time

```
class Calculate: public Parser {
private:
    double x[2], y[2], F[2][2], xx, yy;     // Variables of Bilinear interpolate
    int index_1, index_2;                   // Indices in output_cap and input_trans arrays

    void Get_F(vector<vector<double>> lut_type);    // Extract 2x2 submatrix from LUT
    double bilinear_interpolate();                  // Bilinear interpolate
// Public data for STA to access
public:
    LOAD Calculate_Load(string ZN);    // Compute output load seen by net
    DELAY Calculate_Delay_Trans(string gate_type, double load, double delay);   // Compute delay and transition time
    CellType Get_Cell_Type(const string& name);           // Map gate name to CellType enum
};
```

3. STA 主模組

- 協調其他模組的運作
- 執行遞迴演算法進行 propagation delay
- 輸出分析結果

```
class STA: public Calculate{
private:
    vector<LOAD> gate_load;     // Stores gate load results
    vector<DELAY> gate_delay;   // Stores gate delay results

    PATH longest, shortest;     // Store longest and shortest paths

    void Output_Load();         // Write gate load to file
    void Output_Delay();        // Write gate delay to file
    void Output_Path();         // Write critical paths to file
    void Recursive(string ZN);  // Recursive traversal from output back to inputs

public:
    STA(char* argv[]);          // Constructor to initialize the STA object and parse the files
    void print_gate_and_net();  // Debug print (not used)
    void run();                 // Run STA process
};
```

3.2 Data Structure Design

1. struct Gate

用來描述網表中每個邏輯閘的資訊，包括其型號、輸入與輸出連接的 net 名稱。

```
// represent a logic gate
struct Gate {
    string type;                // Gate type (NOR2X1, INVX1, NAND2X1)
    vector<string> inputs;      // Input nets (A1, A2, I)
    string output;              // Output net (ZN)
};
```

- 每一個 gate 會對應到一筆這樣的資料。
- inputs 中的順序依 gate type 而有所不同(NORX1/NAND2X1 依序為 A1、A2；INVX1 為 I)。

2. struct Net

代表電路中的每一條 net/wire 去追蹤其來源、去處、轉換時間、延遲等。

```
// represent a net/wire
struct Net {
    string type;                // input, output, or wire
    vector<pair<string, string>> gate_in;   // List of gates that use this net as input <gate, pin>
    string gate_out = "";                   // Gate that drives this net
    double trans = 0.0f;                    // Output transition time
    double delay = 0.0f;                    // Output delay time

    PATH worst_case;        // Path object to track worst-case path
};
```

- 每一個 net 會對應到一筆這樣的資料。
- gate_in：記錄它被哪些 gate 當成輸入（可以有多個 fanout）。
- gate_out：記錄是哪一個 gate 的輸出連到這條 net（單一 fanin）。
- trans 和 delay：動態分析過程中遞迴得到的資訊。

3. struct CellLib

用來儲存從.lib 檔中讀出的每一種 cell 的相關參數與 LUT。

```
// Structure to store all data from .lib cell definition
struct CellLib {
    string cell_name;                       // Cell type name
    vector<double> capacitance;             // Pin capacitance list
    vector<vector<double>> cell_rise;
    vector<vector<double>> cell_fall;
    vector<vector<double>> rise_transition;
    vector<vector<double>> fall_transition;
};
```

- 每一個 cell 都對應到 .lib 中一個 cell 區塊。
- LUT 的 x 軸為 output load，y 軸為 input transition。
- LUT 的值用雙線性插值進行查詢。

4. struct LOAD

儲存每個 gate 的 output load 資訊。

```
// store gate output load info
struct LOAD{
    string gate_name;
    double load;
};
```

- 用於輸出至 Load_*.txt，依 load 大小或名字排序。
- 如果此 gate 驅動的是 primary output，固定為 0.03pF。

5. struct DELAY

儲存每個 gate 計算出的 delay 與 transition time。

```
// store gate delay info
struct DELAY{
    string gate_name;
    bool is_rise;  // true: rise, false: fall
    double delay;
    double trans;
};
```

- 用於輸出至 Delay_*.txt，依 delay 大小或名字排序。
- 每個 gate 根據 input transition 與 output load 查表計算 output delay 與 output transition time。
- 根據上升/下降兩者擇其最大作為計算依據。

6. struct PATH

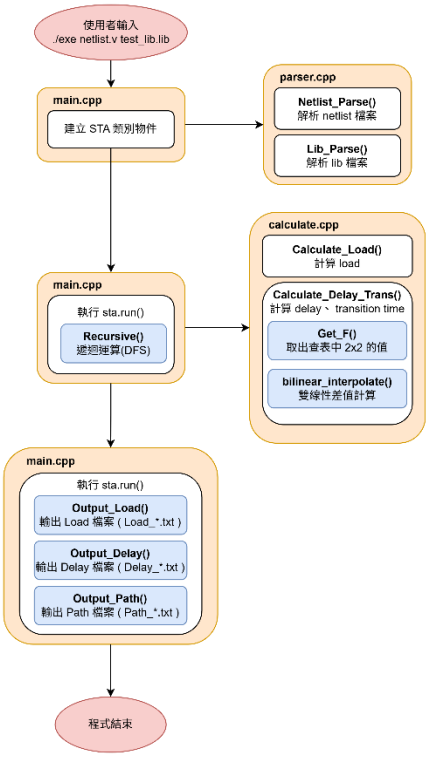
記錄一條從 primary input 到某 net 的路徑及其延遲時間。

```
// represent a path for longest/shortest delay
struct PATH{
    double delay = 0.0f;
    vector<string> path;    // ordered net names along the path
};
```

- 用於輸出至 Path_*.txt。
- 每條 net 都會儲存目前最差的路徑。
- 在遞迴計算過程中更新。
- 最終從所有 output net 中挑出最長與最短者。

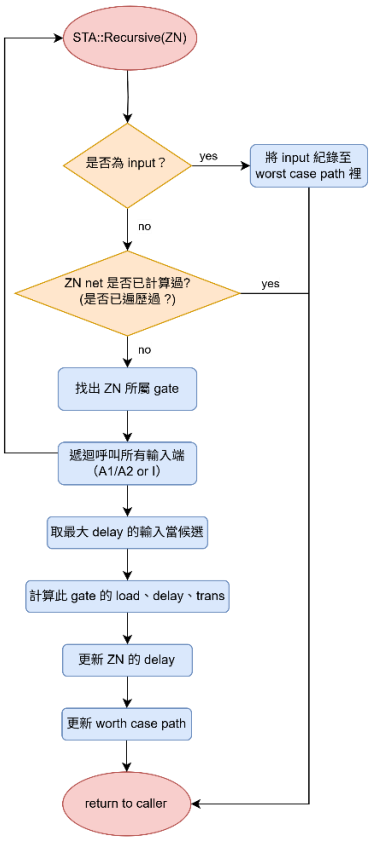
4. Method of Your Design

4.1 Overall Flow Chart



4.2 Core Algorithms

1. STA::Recursive() 流程圖



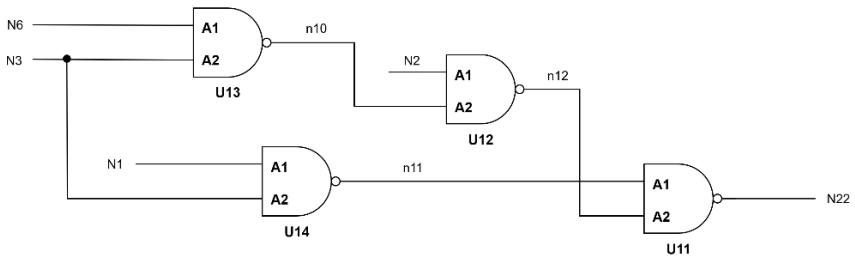
STA::Recursive()為遞迴計算函式，是本系統的主要關鍵，負責對所有 gate 自下而上地：

- 計算 input transition time（選最慢）
- 查表算 delay 與 output transition
- 記錄 critical path

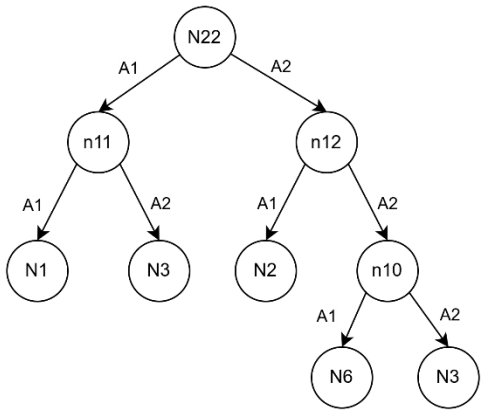
2. 程式碼

```
void STA::Recursive(string ZN){
    // cout << "ZN: " << ZN << " ";
    if(net_map[ZN].type == "input") {
        // cout << endl;
        net_map[ZN].worst_case.path.push_back(ZN);      // Base case: primary input
        return;
    }
    else if(net_map[ZN].delay != 0)
    {
        // cout << endl;
        return;  // if the net is already visited, return
    }
    string gate = net_map[ZN].gate_out;
    double candidate_trans = 0.0f;
    string candidate_path_net;
    // Handle 2-input gates
    if(gate_map[gate].inputs.size() == 2){
        string A1 = gate_map[gate].inputs[0];
        string A2 = gate_map[gate].inputs[1];
        // cout << "A1: " << A1 ;
        // cout << " A2: " << A2 << endl;
        Recursive(A1);      // Process A1
        Recursive(A2);      // Process A2

        // Use the input with higher delay for transition time
        candidate_trans = (net_map[A1].delay >= net_map[A2].delay) ? net_map[A1].trans : net_map[A2].trans;

        // Use the input with the longer path as the path source
        candidate_path_net = (net_map[A1].worst_case.delay >= net_map[A2].worst_case.delay) ? A1 : A2;
    }
    // Handle single-input gates
    else if(gate_map[gate].inputs.size() == 1){
        string I = gate_map[gate].inputs[0];
        PATH path_I;
        // cout << "I: " << I << endl;
        Recursive(I);
        candidate_trans = net_map[I].trans;
        candidate_path_net = I;
    }
    // Calculate the output load
    gate_load.emplace_back(Calculate_Load(ZN));

    // Calculate delay and transition using bilinear interpolation
    gate_delay.emplace_back(Calculate_Delay_Trans(gate, gate_load.back().load, candidate_trans));
    net_map[ZN].trans = gate_delay.back().trans;
    net_map[ZN].delay = gate_delay.back().delay;

    // Update worst-case delay and path
    net_map[ZN].worst_case.delay = net_map[candidate_path_net].worst_case.delay + net_map[ZN].delay;
    // Add wire delay except for primary output
    if(net_map[ZN].type != "output")
        net_map[ZN].worst_case.delay += WIRE_DELAY;
    // Copy the path and append this net
    net_map[ZN].worst_case.path = net_map[candidate_path_net].worst_case.path;
    net_map[ZN].worst_case.path.push_back(ZN);
    return;
}
```

3. 遞迴計算流程圖說明（以 c17.v 中的 N22 為例）



在 STA 過程中，系統會從每一個 primary output 開始，利用 Depth-First Search (DFS) 自下而上的遞迴遍歷。這個過程會探索整個邏輯網路，並對每條 net 計算其 delay 與 transition time，同時記錄從某個 primary input 到該 net 的 worst case delay path。以 N22 為例，其遍歷圖如下:



其遍歷順序為: N22 → n11 → N1 → N3 → n12 → N2 → n10 → N6 →N3。

此順序代表系統會沿著電路的 data path 不斷往前走訪，直到遇到：

- Primary input net（如 N1, N2, N3, N6）→ 因為為輸入節點，不再往前遞迴，直接 return。
- 已計算過的 net → 表示該 net 的 delay 與 transition 已存在於 net_map 中，亦直接 return，避免重複計算。

在每一次遞迴返回的過程中，若當前 gate 的輸入端（input pins）有兩個，例如 NANDX1 的 A1 與 A2，系統會從這兩個輸入端中：

(1) 比較兩條輸入 net 的 delay。

(2) 選擇 delay 較大的輸入 作為此 gate 的有效 input delay（代表最慢來的訊號會主導延遲）。

(3) 根據此 input delay 與此 gate 的 output load，進入 LUT 查表，使用雙線性插值法計算：
   - Output propagation delay
   - Output transition time

接著，會將目前的 net 名稱加入其所屬的 worst case path 中，以建立一條從 primary input 到該 net 的完整延遲路徑資訊。

以 n12 為例，其 gate 為 U12，輸入端為：

- A1 = N2（primary input）
- A2 = n10（由 U13 驅動）

分析步驟如下：

(1) 對 A1 = N2，因為是 input，直接 return。

(2) 對 A2 = n10，為 wire，需繼續遞迴直到 input。

(3) 假設 n10 的 delay 較大，則：

- n10 被視為 n12 的主導輸入。
- 以 n10 的 transition time + n12 的 load 值作為查表參數。
- 查表取得 delay 與 transition time。

(4) 更新 net_map[n12]：

- delay = gate_delay + wire_delay
- trans = 查表 transition time
- path = n10.worst_case.path + n12

這樣一路計算回來，就能準確地在遞迴過程中建立完整的 delay tree 與 path tree。

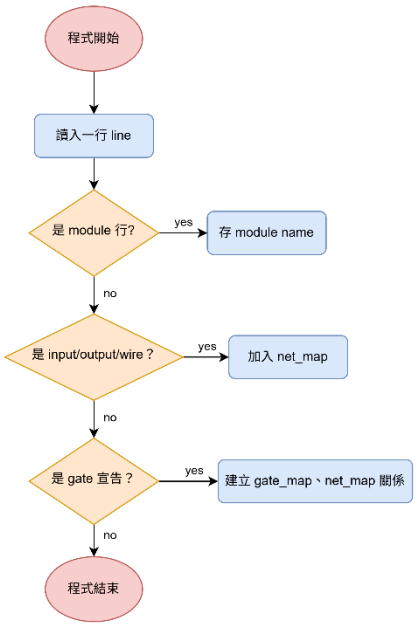
4.3 Implementation Details

1. Parser 模組

此模組負責兩項任務：

- 解析 gate-level Verilog 檔（Netlist_Parse）
- 解析 Liberty 標準 cell library 檔（Lib_Parse）

(1) Parser::Netlist_Parse()

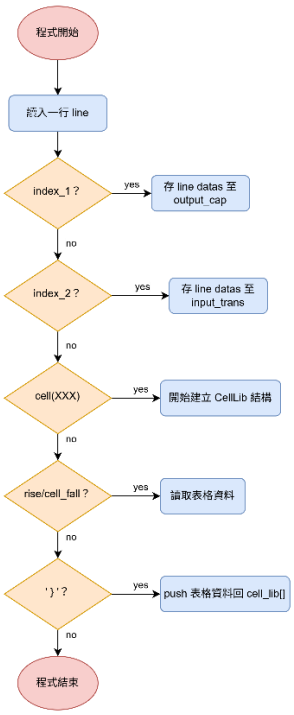


將 netlist 檔案轉成兩個資料結構：

- gate_map<string, Gate>：儲存所有 gate information
- net_map<string, Net>：儲存所有 net 的邏輯關係

```cpp
// Parse Netlist File
void Parser::Netlist_Parse(const string& filename){
    ifstream inputfile(filename, ios::in);
    if(!inputfile)
    {
        cerr<<"File could not be opened"<<endl;
        exit(0);
    }

    // Read file
    string line, type;
    int pin_num = 0;
    while(getline(inputfile, line)){
        // store module name
        if(line.find("//") != string::npos){
            continue;       // Skip comment lines
        }
        else if(line.find("endmodule") != string::npos){
            break;          // End of module
        }
        // Parse module name
        else if(line.find("module ") != string::npos){
            char *token = strtok(&line[0], "\t (.),;");
            token = strtok(NULL, "\t (.),;");
            module_name = token;
        }
        // Parse input/output/wire declarations
        else if(line.find("input") != string::npos || line.find("output") != string::npos || line.find("wire") != string::npos){
            char *token = strtok(&line[0], "\t (.),;");
            type = token;       // Store type (input/output/wire)
            token = strtok(NULL, "\t (.),;");
            while (token != NULL){
                net_map[token].type = type;     // Assign net type
                if(type == "output") output_nets.emplace_back(token);    // Save primary outputs
                token = strtok(NULL, "\t (.),;");
            }
        }
        // Parse logic gate instances and their connections
        else if(line.find("NOR2X1") != string::npos || line.find("INVX1") != string::npos || line.find("NANDX1") != string::npos) {
            char *token = strtok(&line[0], "\t (.),;");
            type = token;           // Gate type (NOR2X1/INVX1/NANDX1)
            token = strtok(NULL, "\t (.),;");
            string gate_name = token;       // ex. U1
            gate_map[gate_name].type = type;    // Store gate type
            token = strtok(NULL, "\t (.),;");
            while (token != NULL){
                string in_out = token;          // Pin name (A1, A2, I, ZN)
                token = strtok(NULL, "\t (.),;");
                if(in_out == "A1" || in_out == "A2" || in_out == "I"){
                    gate_map[gate_name].inputs.emplace_back(token);     // Add input net
                    net_map[token].gate_in.emplace_back(make_pair(gate_name, in_out));      // Update net fanin info
                }
                else if (in_out == "ZN"){
                    gate_map[gate_name].output = token;     // Save output net
                    net_map[token].gate_out = gate_name;    // Connect net to gate output
                }
                token = strtok(NULL, "\t (.),;");
            }
        }
    }
}
```

(2) Parser::Lib_Parse()



解析 .lib，讀取每個 cell 的以下資料：

- pin capacitance
- cell_rise / fall LUT
- rise / fall transition LUT

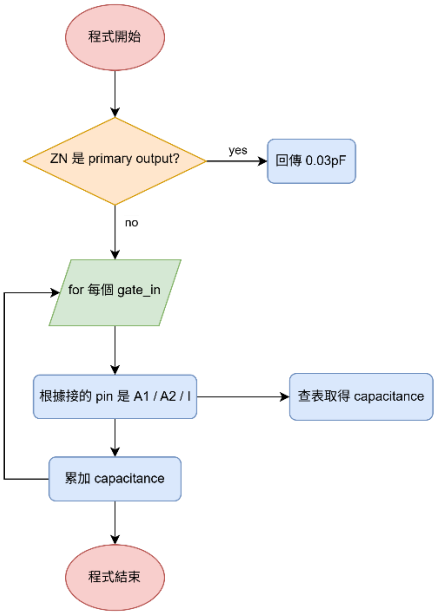
```
// Parse Library File
void Parser::Lib_Parse(const string& filename){
    ifstream inputfile(filename, ios::in);
    if(!inputfile)
    {
        cerr<<"File could not be opened"<<endl;
        exit(0);
    }

    string line;
    CellLib curr_cell;
    bool in_cell = false;
    while(getline(inputfile, line)){
        // store total output net capacitance
        if (line.find("index_1") != string::npos) {
            // cout << "total_output_net_capacitance: ";
            output_cap = Store_Value(line);         // Extract output load index values
        }
        // store input transition time
        else if (line.find("index_2") != string::npos) {
            // cout << "input_transition_time: ";
            input_trans = Store_Value(line);        // Extract input transition index values
        }
        // store cell information
        else if(line.find("cell (") != string::npos){
            in_cell = true;             // Start of cell information
            curr_cell = CellLib();      // Reset current cell
            curr_cell.cell_name = line.substr(line.find("(")+1, line.find(")") - line.find("(") - 1);
            // cout << "Cell: " << curr_cell.cell_name << endl;
        }

        if(in_cell){
            // store cell capacitance
            if(line.find("capacitance") != string::npos){
                string value = line.substr(line.find(":")+2, line.find(";") - line.find(":") - 2);
                curr_cell.capacitance.emplace_back(stof(value));    // Load pin capacitance
                // cout << "capacitance: " << value << endl;
            }
            // store cell_rise
            else if(line.find("cell_rise") != string::npos){
                // cout << "cell_rise: " << endl;
                for(int i=0; i < input_trans.size(); i++){
                    getline(inputfile, line);
                    curr_cell.cell_rise.emplace_back(Store_Value(line));    // Add cell rise
                }
            }
            // store cell_fall
            else if(line.find("cell_fall") != string::npos){
                // cout << "cell_fall: " << endl;
                for(int i=0; i < input_trans.size(); i++){
                    getline(inputfile, line);
                    curr_cell.cell_fall.emplace_back(Store_Value(line));    // Add cell rise
                }
            }
            // store rise_transition
            else if(line.find("rise_transition") != string::npos){
                // cout << "rise_transition: " << endl;
                for(int i=0; i < input_trans.size(); i++){
                    getline(inputfile, line);
                    curr_cell.rise_transition.emplace_back(Store_Value(line));  // Add rise transition
                }
            }
            // store fall_transition
            else if(line.find("fall_transition") != string::npos){
                // cout << "fall_transition: " << endl;
                for(int i=0; i < input_trans.size(); i++){
                    getline(inputfile, line);
                    curr_cell.fall_transition.emplace_back(Store_Value(line));  // Add fall transition
                }
            }
            // end of current cell
            else if(line == "}"){
                cell_lib.emplace_back(curr_cell);  // Store current cell information
                in_cell = false;    // End of current cell
                // cout << "End of cell " << curr_cell.cell_name << endl << endl;
            }
        }

    }
}
```

2. Calculate 模組

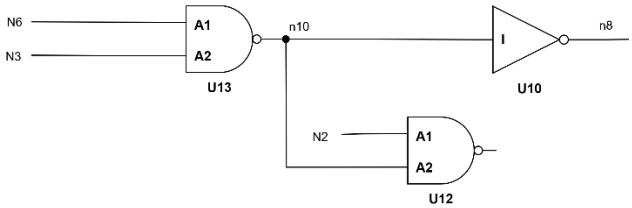
此模組負責與時序有關的數學計算，包括：

- gate 的 output load
- gate 的 propagation delay 與 transition time
- 雙線性插值的計算

(1) Calculate::Calculate_Load()



計算某個 gate 輸出的 net 的負載（即 fanout 的 pin 輸入電容總和）。



以 U13 為例，n10 接到 2 個 gate：

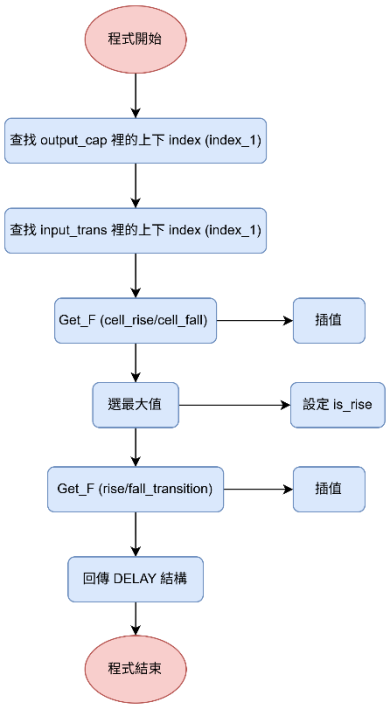
→ I of INVX1 (0.0109115 pF)

→ A2 of NANDX1 (0.00798456 pF)

因此總 load = 0.018896 pF

```
// Compute output load for a net ZN
LOAD Calculate::Calculate_Load(string ZN){
    LOAD value;
    value.gate_name = net_map[ZN].gate_out;     // Identify the gate that drives this net
    value.load = 0.0;
    if(net_map[ZN].type == "output") value.load = OUTPUT_LOAD;      // If net is primary output, use fixed load
    else{
        for(const auto& gate_in : net_map[ZN].gate_in){
            int pidx = (gate_in.second == "A1" || gate_in.second == "I") ? 0 : 1;       // Select pin index
            string type = gate_map[gate_in.first].type;         // Gate type connected to this net
            CellType cidx = Get_Cell_Type(type);                // Lookup cell type
            value.load += cell_lib[cidx].capacitance[pidx];     // Accumulate pin capacitance
        }
    }
    // cout << "gate: " << value.gate_name << " load: " << value.load << endl;
    return value;
}
```

(2) Calculate::Calculate_Delay_Trans()



輸入 gate name、負載、輸入轉換時間

→ 查表並用雙線性插值計算 delay 和 transition

```
// Compute delay and transition time of a gate based on load and input transition
DELAY Calculate::Calculate_Delay_Trans(string gate_name, double load, double delay){
    DELAY value;
    value.is_rise = 1;
    value.gate_name = gate_name;
    // Find bounding indices for interpolation
    auto v1 = lower_bound(output_cap.begin(), output_cap.end(), load);
    auto v2 = lower_bound(input_trans.begin(), input_trans.end(), delay);
    // Ensure to get two valid interpolation points
    x[0] = (load > *(v1 - 1)) ? *(v1 - 1) : *v1;
    x[1] = (load > *(v1 - 1)) ? *v1 : *(v1 + 1);
    y[0] = (delay > *(v2 - 1)) ? *(v2 - 1) : *v2;
    y[1] = (delay > *(v2 - 1)) ? *v2 : *(v2 + 1);
    // Store the actual interpolation point
    xx = load;
    yy = delay;
    // Get indices for accessing LUTs
    index_1 = (load > *(v1 - 1)) ? v1 - output_cap.begin() - 1 : v1 - output_cap.begin();
    index_2 = (delay > *(v2 - 1)) ? v2 - input_trans.begin() - 1 : v2 - input_trans.begin();
    // Lookup cell type
    CellType cidx = Get_Cell_Type(gate_map[gate_name].type);

    // Interpolate cell_rise
    Get_F(cell_lib[cidx].cell_rise);
    double delay_value1 = bilinear_interpolate();

    // Interpolate cell_fall
    Get_F(cell_lib[cidx].cell_fall);
    double delay_value2 = bilinear_interpolate();

    // Use larger of rise/fall delay
    if(delay_value1 < delay_value2) {
        delay_value1 = delay_value2;
        value.is_rise = 0;
    }
    value.delay = delay_value1;

    // Interpolate corresponding transition
    if(value.is_rise == 1)
        Get_F(cell_lib[cidx].rise_transition);
    else
        Get_F(cell_lib[cidx].fall_transition);
    value.trans = bilinear_interpolate();

    // cout << value.gate_name << " " << value.is_rise << " " << value.delay << " " << value.trans << endl;
    return value;
}
```

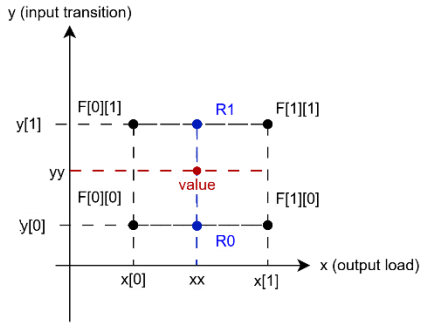
(3) Calculate::Get_F()

從一張 LUT 中擷取 2x2 格點作為插值基礎

```
// Extract 2x2 values from LUT for interpolation
void Calculate::Get_F(vector<vector<double>> lut_type){
    F[0][0] = lut_type[index_2][index_1];           // Top-Left
    F[1][0] = lut_type[index_2][index_1 + 1];       // Top-right
    F[0][1] = lut_type[index_2 + 1][index_1];       // Bottom-Left
    F[1][1] = lut_type[index_2 + 1][index_1 + 1];   // Bottom-right
}
```

(4) Calculate::bilinear_interpolate()

根據 2x2 LUT 值與輸入 (xx, yy)，計算插值結果。

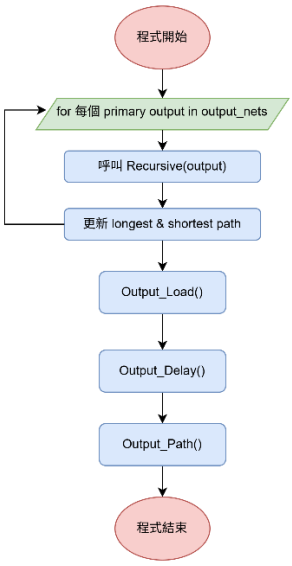



```
// Compute output load for a net ZN
LOAD Calculate::Calculate_Load(string ZN){
    LOAD value;
    value.gate_name = net_map[ZN].gate_out;     // Identify the gate that drives this net
    value.load = 0.0;
    if(net_map[ZN].type == "output") value.load = OUTPUT_LOAD;      // If net is primary output, use fixed load
    else{
        for(const auto& gate_in : net_map[ZN].gate_in){
            int pidx = (gate_in.second == "A1" || gate_in.second == "I") ? 0 : 1;       // Select pin index
            string type = gate_map[gate_in.first].type;         // Gate type connected to this net
            CellType cidx = Get_Cell_Type(type);                // Lookup cell type
            value.load += cell_lib[cidx].capacitance[pidx];     // Accumulate pin capacitance
        }
    }
    // cout << "gate: " << value.gate_name << " load: " << value.load << endl;
    return value;
}
```

3. STA 模組

此模組為主控制器，呼叫所有其他模組完成 STA。

(1) STA::run()



主要程式：對每個 primary output 做遞迴 timing analysis，並產生三份輸出檔。

(2) STA::Output_Load()

將 gate_load 依照 load 值排序後輸出至檔案。若 load 值一樣，則依照名稱做排序。

```
// Output gate load values to file
void STA::Output_Load(){
    // name the output file
    string filename = "Load_110501544_";
    filename += module_name + ".txt";
    // open the output file
    ofstream outfile(filename);
    if(!outfile){
        cerr << "cannot open output file: " << filename << endl;
        exit(1);
    }
    // sort the gate load by load value and gate name
    sort(gate_load.begin(), gate_load.end(), cmp_load);
    // output the gate load to the file
    for(const auto& gate : gate_load){
        outfile << gate.gate_name << " " << fixed << setprecision(6) << gate.load << endl;
    }
    outfile.close();
}
```

(3) STA::Output_Delay()

將所有 gate 的 delay 資訊依 delay 值排序後輸出。若 delay 值一樣，則依照名稱做排序。

```
// Output gate delay and transition time to file
void STA::Output_Delay(){
    // name the output file
    string filename = "Delay_110501544_";
    filename += module_name + ".txt";
    // open the output file
    ofstream outfile(filename);
    if(!outfile){
        cerr << "cannot open output file: " << filename << endl;
        exit(1);
    }
    // sort the gate load by load value and gate name
    sort(gate_delay.begin(), gate_delay.end(), cmp_delay);
    // output the gate load to the file
    for(const auto& gate : gate_delay){
        outfile << gate.gate_name << " " << gate.is_rise << " " << fixed << setprecision(6) << gate.delay << " " << gate.trans << endl;
    }
    outfile.close();
}
```

(4) STA::Output_Path()

輸出最長與最短 path 的 delay 與經過的點。

```
// Output longest and shortest timing paths to file
void STA::Output_Path(){
    string filename = "Path_110501544_";
    filename += module_name + ".txt";
    // open the output file
    ofstream outfile(filename);
    if(!outfile){
        cerr << "cannot open output file: " << filename << endl;
        exit(1);
    }

    // Longest path
    outfile << "Longest delay = " << fixed << setprecision(6) << longest.delay;
    outfile << ", the path is: ";
    for (size_t i = 0; i < longest.path.size(); ++i) {
        outfile << longest.path[i];
        if (i != longest.path.size() - 1)
            outfile << " -> ";
    }
    outfile << endl;

    // Shortest path
    outfile << "Shortest delay = " << fixed << setprecision(6) << shortest.delay;
    outfile << ", the path is: ";
    for (size_t i = 0; i < shortest.path.size(); ++i) {
        outfile << shortest.path[i];
        if (i != shortest.path.size() - 1)
            outfile << " -> ";
    }
    outfile.close();
}
```

5. Hardness

1. netlist 檔的解析

原本以為把 gate-level netlist 讀進來應該很簡單，沒想到 Verilog 檔案的格式其實滿彈性的，有時候一行好幾個東西，有時又有註解、空白、換行什麼的，光是把一行正確解析成「這是哪個 gate、有哪些 pin」就搞超久。

例如下面的這兩段：

```
NOR2X1 U8 (.A1(n8), .A2(n9), .ZN(N23));    → c17.v
NANDX1 g3(.ZN(n6), .A1(n4), .A2(n5));      → example.v
```

U8 後面有空格，然而 g3 卻沒有，如果用 operator (>>)讀檔，很容易讀取錯誤。因此我最後使用 getline()的方式去讀檔，並用 strtok() 把所有奇怪的符號都先清乾淨，再一個 token 一個 token 慢慢 parse 才成功。

2. 雙線性插值的邏輯

雖然說雙線性插值這東西微積分有教過，但真的要實作還是卡了一陣子。主要是要先抓到 LUT 的哪四個點（左上、右上、左下、右下），然後根據目前的輸入值做插值，公式實際上就是線性差值做兩次，然而在寫程式的時候有些搞不懂，後來慢慢看影片教學才了解。此外，我一開始抓錯 index，結果跑出來的 delay 都怪怪的，還以為是 lib 檔有問題，debug 了半天才發現是 index_1 和 index_2 搞反了。後來就用 lower_bound() 配合條件判斷，先找到正確的 index，再用插值公式一步一步算出來，才讓整個 timing 分析變得合理。

3. 遞迴傳播 delay 的順序

STA 最核心的部分是從 output net 開始往回追，把每個 gate 的輸入都先算好 delay，再往後算下一層，但這個順序真的很不直覺。

一開始我用迴圈去跑，但發現無法保證所有 input 都算好了，只能改成遞迴。遞迴本身不難寫，但要把每個 net 的 delay、transition、path 都記住還要避開重算（已經算過的 net 要跳過），真的滿燒腦的。試了幾種資料結構之後，最後決定在 net_map 裡面加上一個 delay != 0 的判斷，才讓整個流程順順的跑完。

6. Suggestions

對初學者來說，像這次作業其實不只是寫出功能，也是在練寫一個「能維護、能看懂的程式」。但有時候不知道該怎麼切檔案、怎麼設計 class。

如果老師可以提供一份範例架構，如：

- 建議幾個 .cpp / .hpp 的切分方式
- 資料要封裝在哪裡、怎麼寫才不會太亂

這樣可以讓我們不只有會寫功能，也會慢慢培養好架構的能力。